

Software Engineering

Function-Oriented Software Design

(Lecture 7b)

15-09-2026

Dr. Prahlada Rao B B

ANRF PM Professor

File: NIT-CS3501- L7b- Function-Oriented Software Design- Prahlad

CSE Department

National Institute of Technology Manipur- Imphal 1

Decompose How Long?

- **Decomposition of a bubble** should be **carried on until a level** at which **function of the bubble**:
 - can be **represented** using a **simple set of instructions**.
- For simple problems **Decomposition upto level 1** should suffice.
- However for **Large industry standard problems** may need **Decomposition upto level 3** or **level 4**.
- Rarely if **ever Decomposition beyond level 4** be needed.

Exempl3: Trading-House Automation System (TAS)

- **A large trading house wants us to develop a software:**
 - To automate book keeping activities associated with its business.
- **It has many regular customers:**
 - Who place orders for various kinds of commodities.

3

Example 3: Trading-House Automation System (TAS)

- **The trading house maintains names and addresses of its regular customers.**
- **Each customer is assigned a unique customer identification number (CIN).**
- **As practice when a customer places order:**
 - The accounts department first checks the credit-worthiness of the customer.

4

Example: Trading-House Automation System (TAS)

- The **credit worthiness** of a customer is determined:
 - By analyzing the **history of his payments** to the **bills sent** to him in the past.
- If a customer is **not credit-worthy**:
 - His **orders** are not processed any further
 - An appropriate **order rejection message** is generated for the **customer**.

5

Example: Trading-House Automation System (TAS)

- If a customer is **credit-worthy**:
 - Items he/she **has ordered** are **checked** against the **list of items** the trading house **deals** with.
- The **items** that the **trading house** does not deal with:
 - Are **not processed** any further
 - An appropriate **message for the customer** for these items is generated.

6

Example: Trading-House Automation System (TAS)

- **The items in a customer's order that the trading house deals with:**
 - Are checked for availability in inventory.
- **If the items are available in the inventory in desired quantities:**
 - A bill with the forwarding address of the customer is printed.
 - A material issue slip is printed.

7

Example: Trading-House Automation System (TAS)

- **The customer can produce the material issue slip at the store house:**
 - Take delivery of the items.
 - Inventory data adjusted to reflect the sale to the customer.

8

Example: Trading-House Automation System (TAS)

- If an ordered item is not available in the inventory in sufficient quantity:
 - To be able to fulfill pending orders store details in a "pending-order" file :
 - out-of-stock items along with quantity ordered.
 - customer identification number

9

Ex: Trading-House Automation System (TAS)

- The purchase department would periodically issue commands
 - to generate indents.
- When generate indents command is issued:
 - The system should examine the "pending-order" file
 - Determine the orders that are pending
 - Total quantity required for each of the items.

10

Ex: Trading-House Automation System (TAS)

- **TAS should find out the addresses of the vendors who supply the required items:**
 - **Examine the file containing vendor details** (their address, items they supply etc.)
 - **Print out indents to those vendors.**

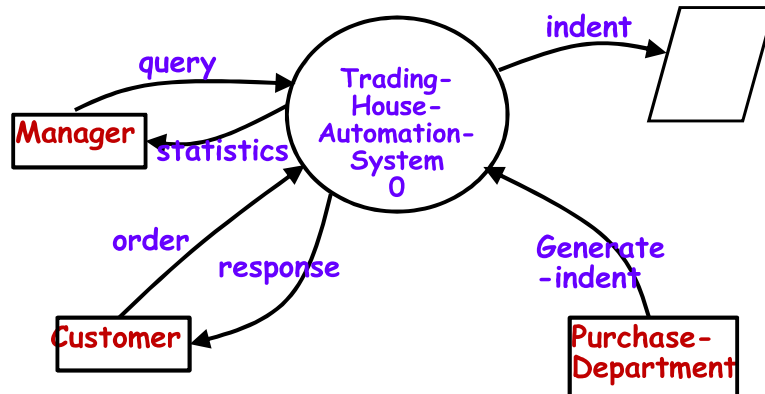
11

Example: Trading-House Automation System (TAS)

- **TAS should also answers managerial queries:**
 - **Statistics of different items sold over any given period of time**
 - **Corresponding quantity sold and the price realized.**

12

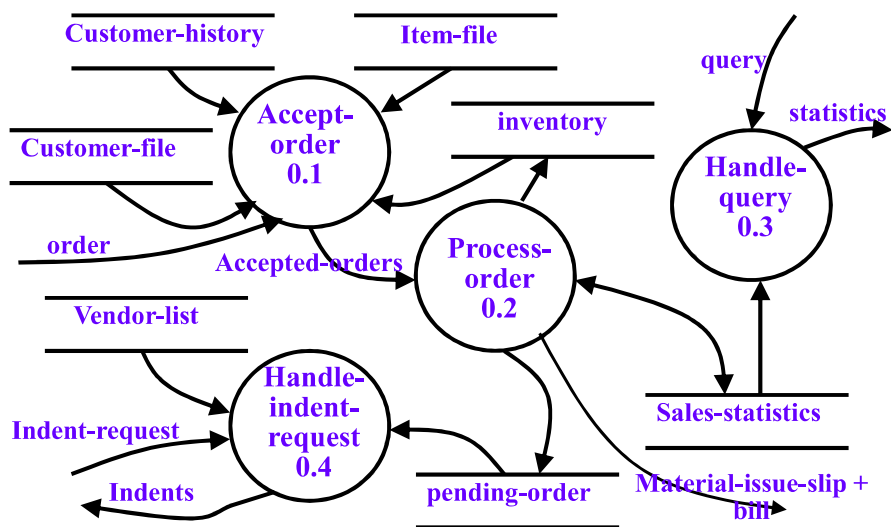
Example-TAS: Context Diagram



Level 0 Context Diagram of TAS

13

Example-TAS: Level 1 DFD



Level 1 DFD of TAS

14

Example-TAS: Data Dictionary

- response: [bill + material-issue-slip, reject-message]
- query: period /* query from manager regarding sales statistics*/
- period: [date+date,month,year,day]
- date: year + month + day
- year: integer
- month: integer
- day: integer
- order: customer-id + {items + quantity}*
- accepted-order: order /* ordered items available in inventory */
- reject-message: order + message /* rejection message */
- pending-orders: customer-id + {items+quantity}*
- customer-address: name+house#+street#+city+pin

15

Example-TAS: Data Dictionary

- item-name: string
- house#: string
- street#: string
- city: string
- pin: integer
- customer-id: integer
- bill: {item + quantity + price}* + total-amount + customer-address
- material-issue-slip: message + item + quantity + customer-address
- message: string
- statistics: {item + quantity + price }*
- sales-statistics: {statistics}*
- quantity: integer

16

Observation

- From the examples,
 - Observe that DFDs help create:
 - Data model
 - Function model

17

Observation

- As a DFD is refined into greater levels of detail:
 - The analyst performs an implicit functional decomposition.
 - At the same time, refinements of data takes place.

18

Guidelines for Constructing DFDs

- **Context diagram should represent the system as a single bubble:**
 - Many beginners commit the mistake of drawing more than one bubble in the context diagram.

19

Guidelines for Constructing DFDs

- **All external entities should be represented in the context diagram:**
 - External entities should not appear at any other level of DFD.
- **Only 3 to 7 bubbles per diagram should be allowed:**
 - Each bubble should be decomposed to between 3 and 7 bubbles.

20

Guidelines For Constructing DFDs

- **A common mistake committed by many beginners:**
 - Attempting to **represent control information** in a **DFD**.
 - **e.g. trying to represent the order** in which different **functions are executed**.

21

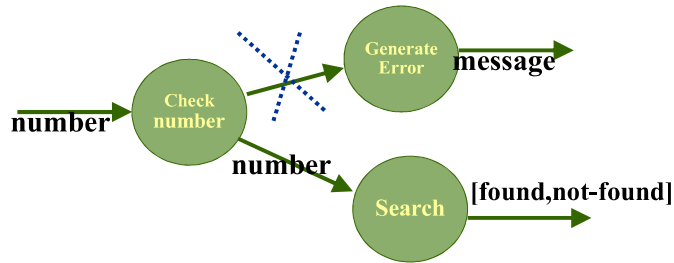
Guidelines For Constructing DFDs

- **A DFD does not represent control information:**
 - **When** or in **what order** different **functions (processes)** **are invoked**
 - **The conditions** under which different **functions are invoked are not represented**.
- **For example, a function might invoke one function or another depending on some condition.**
 - × **Many beginners try to represent this aspect by drawing an arrow between the corresponding bubbles.**

22

Example-1

- **Check the input value:**
 - **If the input value is < -1000 or $> +1000$ generate an error message**
 - **otherwise search for the number**



23

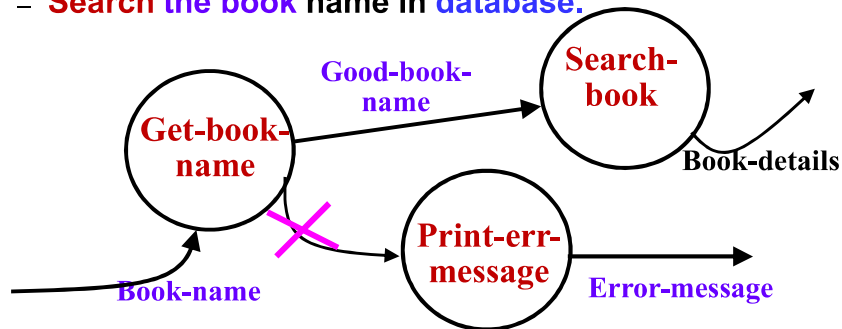
Guidelines For Constructing DFDs

- **If a bubble A invokes either bubble B or bubble C depending on some conditions:**
 - **represent the data that flows from bubble A to bubble B and bubbles A to C**
 - **not the conditions depending on which process is invoked.**

24

Example-2

- A function accepts the book name to be searched from the user
- If the entered book name is not a valid book name
 - Generates an error message,
- If the book name is valid,
 - Search the book name in database.



25

Guidelines For Constructing DFDs

- All functions of the system must be captured in the DFD model:
 - No function specified in the SRS document should be overlooked.
- Only the functions specified in SRS document should be represented:
 - Do not assume extra functionality of the system not specified by the SRS document.

26

Commonly Made Errors

- Unbalanced DFDs
- Forgetting to mention the names of the data flows
- Unrepresented functions or data
- External entities appearing at higher level DFDs
- Trying to represent control aspects
- Context diagram having more than one bubble
- A bubble decomposed into too many bubbles in the next level
- Terminating decomposition too early
- Nouns used in naming bubbles

27

Shortcomings of DFD Model

- DFD models suffer from several shortcomings:
- DFDs leave ample scope to be imprecise.
 - In a DFD model, we infer about the function performed by a bubble from its label.
 - A label may not capture all the functionality of a bubble.

28

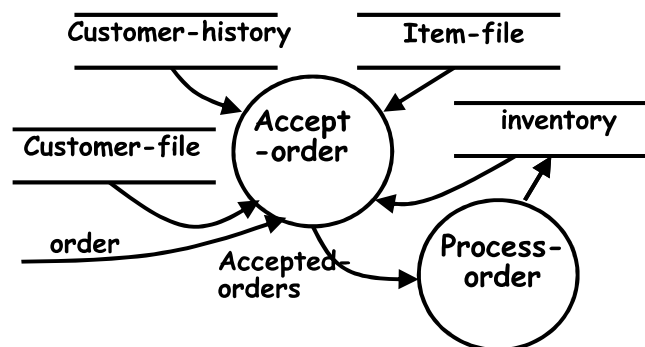
Shortcomings of the DFD Model

- For ex, a bubble named find-book-position has only intuitive meaning:
- Does not specify several things:
 - What happens when some input information is missing or is incorrect.
 - Does not convey anything regarding what happens when book is not found
 - or What happens if there are books by different authors with the same book title.

29

Shortcomings of the DFD Model

- Control information is not represented:
 - For instance, order in which inputs are consumed and outputs are produced is not specified.



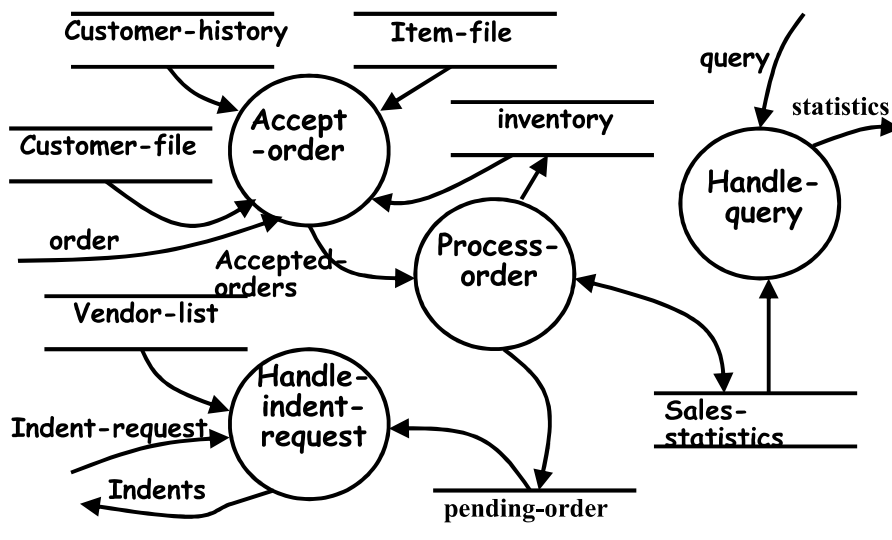
30

Shortcomings of the DFD Model

- **A DFD does not specify synchronization aspects:**
- For instance, **the DFD in TAS example does not specify:**
 - Whether **process-order** may wait until the **accept-order** produces data
 - Whether **accept-order** and **handle-order** may **proceed simultaneously** with some **buffering mechanism** between them.

31

TAS: Level 1 DFD



32

Shortcomings of the DFD Model

- The way decomposition is carried out to arrive at the successive levels of a DFD is subjective.
- The ultimate level to which decomposition is carried out is subjective:
 - Depends on the choice and judgement of the analyst.
- Even for the same problem,
 - Several alternative DFD representations are possible:
 - Many times it is not possible to say which DFD representation is superior or preferable.

33

Shortcomings of the DFD Model

- DFD technique does not provide:
 - Any clear guidance as to how exactly one should go about Decomposing a function:
 - One has to use subjective judgement to carry out Decomposition.
- Structured Analysis techniques do not specify when to stop a decomposition process:
 - To what length Decomposition needs to be carried out.

34

Ext'd DFD Technique to Real-Time Systems

- For real-time systems (having time bounds on actions)
 - Essential to model control flow and events.
- Widely accept'd Tech: Ward and Mellor Technique:
 - A type of process (bubbles) that handle only control flows is introduced.
 - These processes are represented using dashed circles.
 - Separated Data Flow Diagram and Control Flow Diagrams (CFD)
 - To link DFD and CFD a notional ref (solid bar) to control specification is used.
 - CSPEC describes the following:
 - Effect of External event or control Signal
 - Processes that are invoked as a consequence of an event

35

Structured Design

- The aim of structured design is to
 - Transform the results of structured analysis (i.e., a DFD model) into a Structure Chart.
- A Structure Chart represents the software architecture:
 - Various modules making up the system,
 - Module dependency (i.e. which module calls which other modules),
 - Parameters passed among different modules.

36

The Structure Chart

- **The Structure Chart representation is**
 - Easily implementable using programming languages.
- **Main focus of a Structure Chart repr. is on:**
 - Define the module structure of a software,
 - Interaction among different modules,
 - Procedural aspects (e.g, how a particular functionality is achieved) are not represented.

37

Building Blocks of Structure Chart

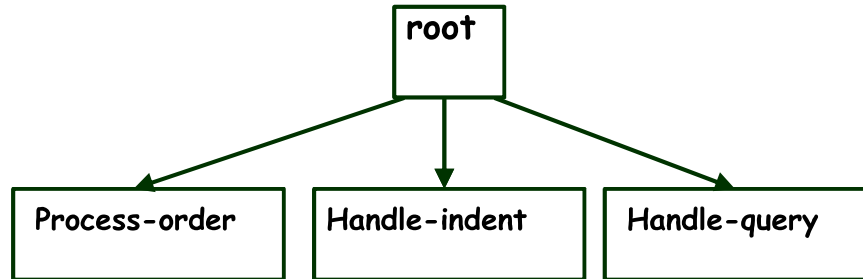
- **Rectangular box:**
 - A rectangular box represents a module.
 - Annotated with the name of the module it represents.

Process-order

38

Module Invocation Arrows

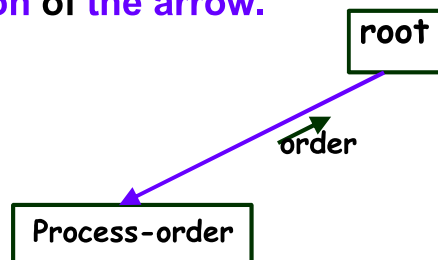
- **An arrow between two modules implies:**
 - **During execution control is passed from one module to the other in the direction of the arrow.**



39

Data Flow Arrows

- **Data flow arrows represent:**
 - These are **small arrows** appearing **alongside** **module invocation arrows**.
 - **Data passing** from **one module** to **another** in the **direction** of the arrow.



40

Library Modules

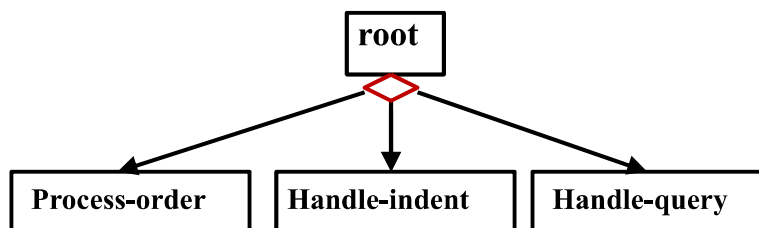
- **Library modules** represent frequently called modules:
 - A rectangle with double side edges.
 - Simplifies drawing when a module is called by several modules.



41

Selection- Symbol

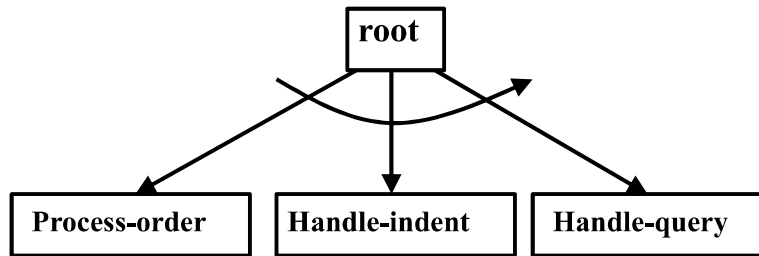
- **The Diamond Symbol** represents:
 - One module of several modules connected to the diamond symbol is invoked depending on some condition.



42

Repetition

- A loop around control flow arrows denotes that the concerned modules are invoked repeatedly.



43

Structure Chart

- There is only one module at the top:
 - the root module.
- There should be at most one control relationship bet'n any two modules:
 - if module A invokes module B,
 - module B cannot invoke module A.
- The main reason behind this restriction is to:
 - consider modules in a Structure Chart to be arranged in layers or levels.

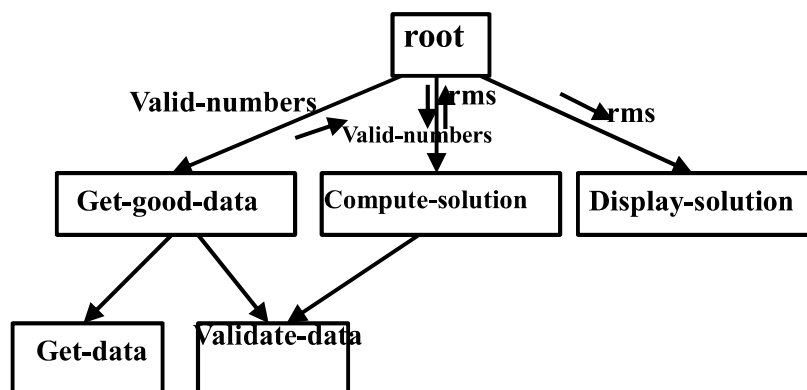
44

Structure Chart

- **The principle of abstraction:**
 - **does not allow lower-level modules to invoke higher-level modules:**
 - **But, two higher-level modules can invoke the same lower-level module.**

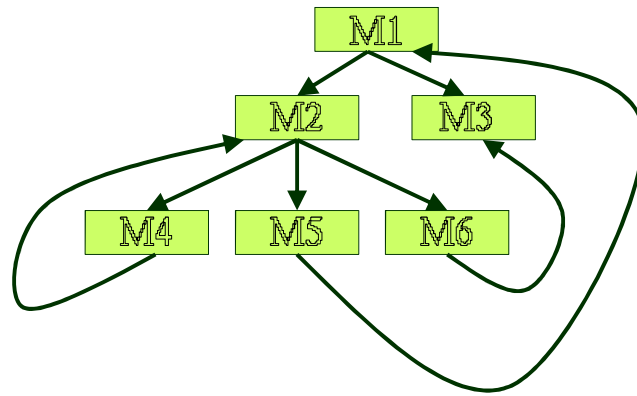
45

Ex: of Properly layered Design-Structure Chart



46

Ex: of Poorly layered Design-Structure Chart



47

Shortcomings of Structure Chart

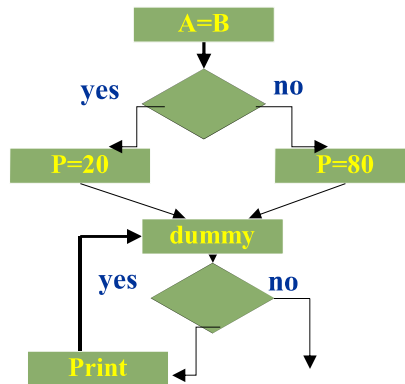
- **By looking at a structure chart:**
 - we can not say whether a module calls another module just once or many times.
- **Also, by looking at a structure chart:**
 - we can not tell the order in which the different modules are invoked.

48

Flow Chart (Aside)

- We are all familiar with the flow chart representations:
 - Flow chart is a convenient technique to represent the flow of control in a system.

```
A=B
if(c == 100)
    P=20
else p= 80
while(p>20)
    print(student mark)
```



49

Flow Chart versus Structure Chart

- A structure chart differs from a flow chart in three principal ways:
 - It is difficult to identify modules of a software from its flow chart representation.
 - Data interchange among the modules is not represented in a flow chart.
 - Sequential ordering of tasks i.e inherent in a flow chart is suppressed in a structure chart.

50

6.5 Transformation of a DFD Model into Structure Chart

- Techniques available to transform a DFD repr of a prob into Structure Chart Model:
- Two strategies exist to guide Transformat'n of a DFD into a Structure Chart:
 - Transform Analysis
 - Transaction Analysis
- Normally, one starts with the level 1 DFD, transforms it into module representation using either The Transform or The Transaction Analysis and then proceeds towards the lower-level DFDs.
- At each level of Transformation, it is important to first determine whether The Transform or The Transaction Analysis is applicable to a particular DFD.

51

Transform Analysis

First step in Transform analysis:

- Divide the DFD into 3 parts:
 - Input,
 - Logical Processing,
 - Output.

52

Transform Analysis

- **Input portion in the DFD:**
 - **Processes** which convert **input data** from **physical** to **logical form**.
 - **E.G.** Read **characters** (physical) from the **terminal** and **store** in internal **tables** or **lists** (logical form)
- **Each Input portion:**
 - called an **afferent branch**.
 - Possible to have **more than one afferent branch** in a **DFD**.

53

Transform Analysis

- **Output portion in the DFD:**
 - **Transforms output data** from **logical form** to **physical form**.
 - **Each output portion:**
 - called an **efferent branch**.
- **The remaining portions of a DFD**
 - called **central transform**

54

Central Transform

second step in Transform analysis:

- Derive structure chart by drawing one functional component for the Central Transform and the
 - each afferent branch and
 - each efferent branches

55

Transform Analysis

- Identifying the highest level input and output transforms requires experience and skill.
- One possible approach is to trace the inputs until a bubble is found whose output cannot be deduced from its inputs alone.
- Processes which validate input or add information to them are not Central Transforms.
- Processes which sort input or filter data from it are Central Transforms.
- First level structure chart is produced by representing each input and output unit as boxes and each central transform as a single box.

56

Transform Analysis

- Identifying the highest level input and output transforms requires experience and skill.
- One possible approach, or guidelines:
 - Trace the inputs until a bubble is found whose output cannot be deduced from the inputs alone.
 - Processes which validate input are not central transforms.
 - Processes which sort input or filter data from it are.
- First level structure chart is produced by representing each input and output unit as boxes and each central transform as a single box.

57

Transform Analysis

- First level of structure chart:
 - Draw a box for each input and output units
 - A box for the central transform.
- Next, refine the structure chart:
 - Add sub-functions required by each high-level module.
 - Many levels of modules may required to be added.

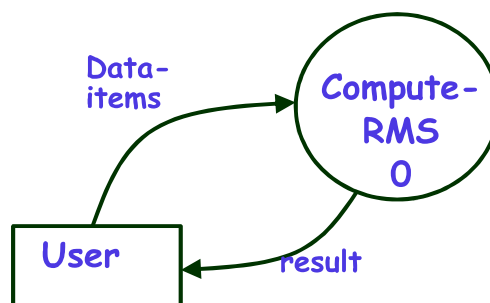
58

Factoring

- The process of breaking functional components into sub-components.
- Factoring includes adding:
 - Read and write modules,
 - Error-handling modules,
 - Initialization and termination modules, etc.
- Finally check:
 - Whether all bubbles have been mapped to modules.

59

Example 1: RMS Calculating Software



Context Diagram

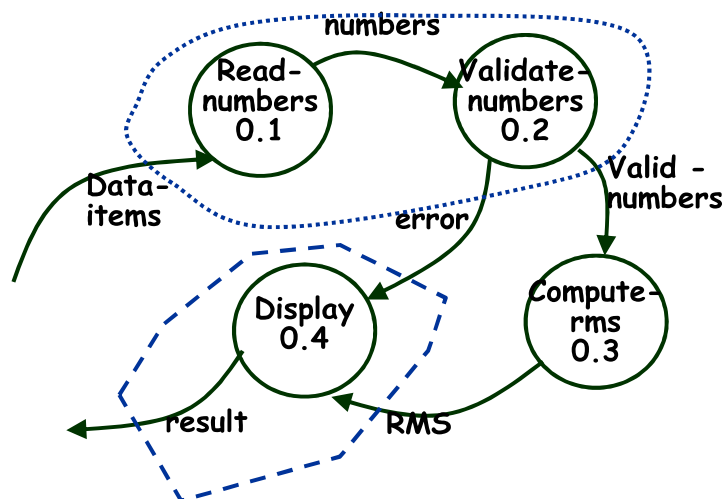
60

Example 1: RMS Calculating Software

- From a cursory analysis of problem description
- Easy to see that the system needs to perform:
 - Accept the input numbers from the user,
 - Validate the numbers,
 - Calculate the root mean square of the input numbers,
 - Display the result.

61

Example 1: RMS Calculating Software



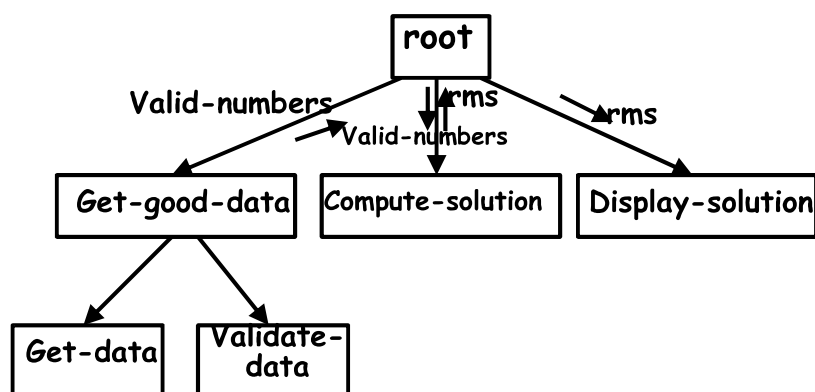
62

Example 1: RMS Calculating Software

- By observing the **level-1 DFD**:
 - **Identify read-number** and **validate-number bubbles** as the **afferent branch**
 - **Display** as the **efferent branch**.

63

Example 1: RMS Calculating Software



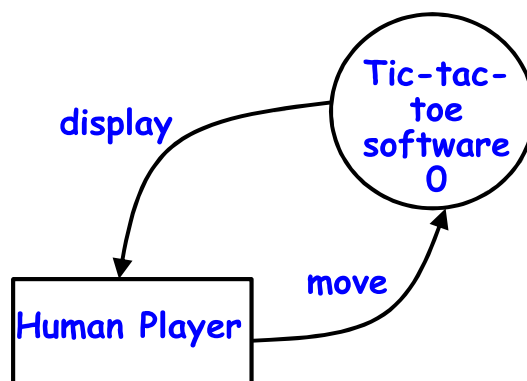
64

Example 2: Tic-Tac-Toe Computer Game

- As soon as either of the human player or the computer wins,
 - A message congratulating the winner should be displayed.
- If neither player manages to get three consecutive marks along a straight line,
 - And all the squares on the board are filled up,
 - Then the game is drawn.
- The computer always tries to win a game.

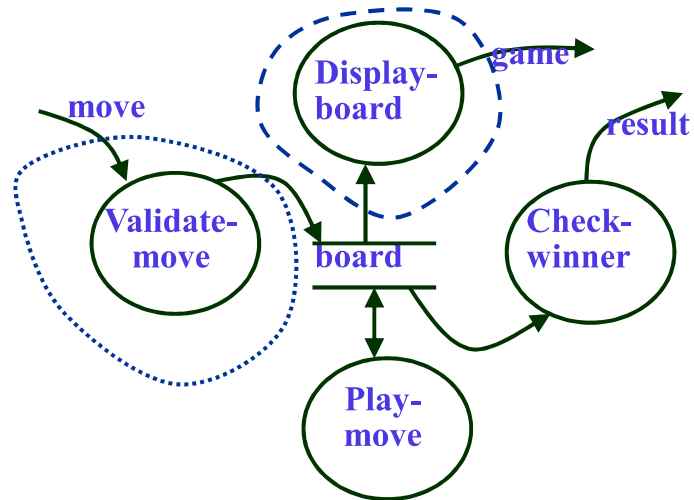
65

Context Diagram for Example 2



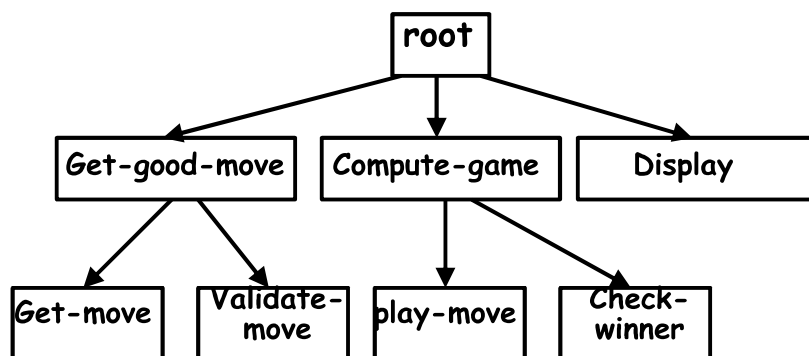
66

Level 1 DFD



67

Structure Chart



68

Transaction Analysis

- **Useful for designing transaction processing programs.**
- **Transform-centered systems:**
 - Characterized by ‘**similar processing steps for every data item**’ processed by **input, process, and output bubbles.**
- **Transaction-driven systems,**
 - ‘**One of several possible paths**’ through the **DFD** is traversed **depending** upon the **input data value.**

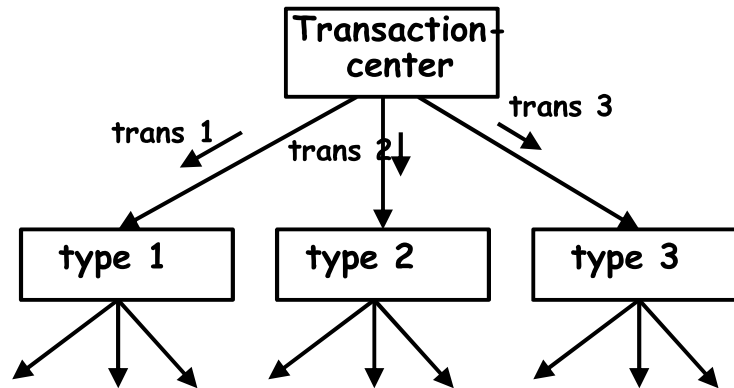
69

Transaction Analysis

- **Transaction:**
 - Any **input data value** that triggers an action:
 - **For ex**, selected **menu options** might trigger different functions.
 - Represented by a **tag** identifying its type.
- **Transaction Analysis** uses this **tag** to divide the system into:
 - **Several transaction modules**
 - **One transaction-center module.**

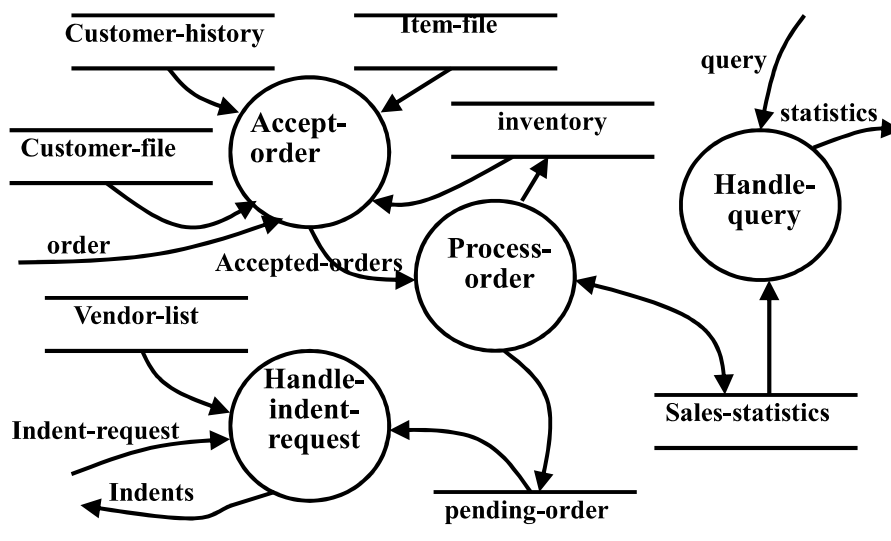
70

Transaction analysis



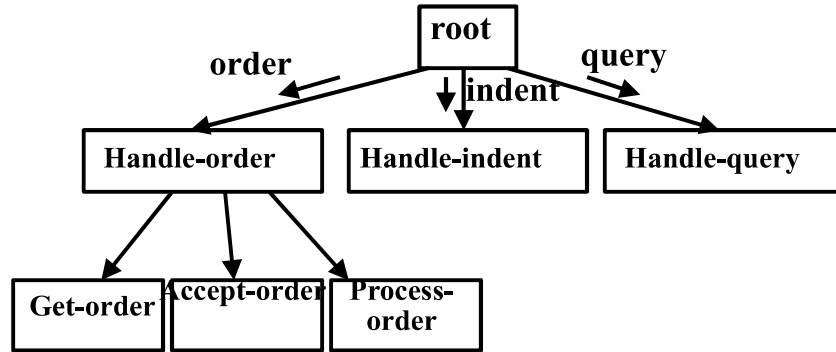
71

Level 1 DFD for TAS



72

Structure Chart



73

Summary

- First discussed **structured analysis** of a larger problem.
- We defined some general guidelines
 - for constructing a **satisfactory DFD model**.
- **The DFD model** though simple and useful
 - Does have several short comings.
- We then started discussing **Structured Design**.
- **Aim of Structured Design:**
 - Transform a **DFD** representation into a **Structure Chart**.
- **Structure Chart** represents:
 - **Module structure**
 - **Interaction among different modules,**
 - **Procedural aspects are not represented.**

74

Summary

- **Structured Design provides two strategies to Transform a DFD into a Structure Chart:**
 - Transform Analysis
 - Transaction Analysis
- We discussed **three examples of Structured Design.**
- It takes a lot of **practice** to become a **good software designer:**

18-09-2026

75

Thank Q

18-09-2026

76